RL for LLM

Teaching Large Language Models to Reason, Align, and Improve with Reinforcement Learning

Language Reward Learn Align

by **Elik Sror**



The Beginning...

*"We trained this model using Reinforcement Learning from Human Feedback (RLHF), using the same methods as **InstructGPT** , but with slight differences in the data collection setup. We trained an initial model using supervised fine-tuning: human AI trainers provided conversations in which they played both sides—the user and an AI assistant. We gave the trainers access to model-written suggestions to help them compose their responses."*

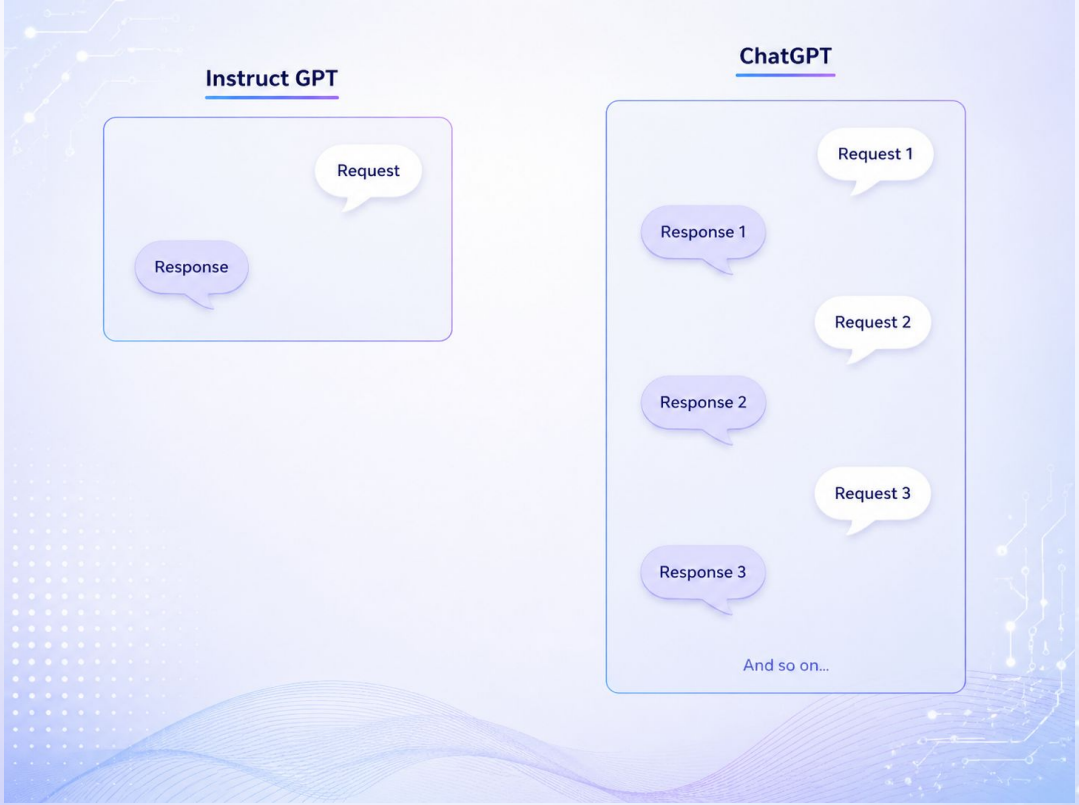
From OpenAI's original ChatGPT launch article

Training language models to follow instructions with human feedback (3/2022)



- *Fundamentally, GPT-3 is just a language model.*
    - *When given a piece of text (i.e. prompt), it generates, somewhat randomly, plausible subsequent text that fits the best.*
- *This objective is misaligned with the goal of "following the user's instructions helpfully and safely."*
- ***InstructGPT*** *focuses on how to effectively align large language models such as GPT-3 with user intent through fine-tuning with human feedback so that the output can be more* ***helpful, truthful and harmless.***

Instruct GPT vs. ChatGPT



How does the magic work?

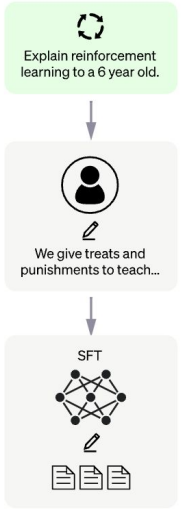
Step 1

Collect demonstration data and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3.5 with supervised learning.



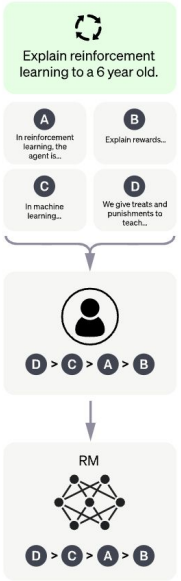
Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

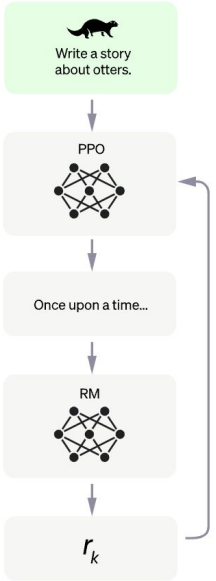
A new prompt is sampled from the dataset.

The PPO model is initialized from the supervised policy.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Agenda for today

01 Pre-training

02 Fine tuning

03 RL Recap

04 RLHF Recap

05 <u>Our main subject for today:</u>
PPO - Proximal Policy Optimization (~2019)
+ GAE - Generalized Advantage Estimation

Abbreviations for today :)

- SL - Supervised Learning
- KL - Kullback-Leibler divergence
- MDP - Markov decision process
- PPO - Proximal Policy Optimization
- TRPO - Trust Region Policy Optimization
- GAE - Generalized Advantage Estimation
- TD - Temporal Difference
- GRPO - Group Relative Policy Optimization

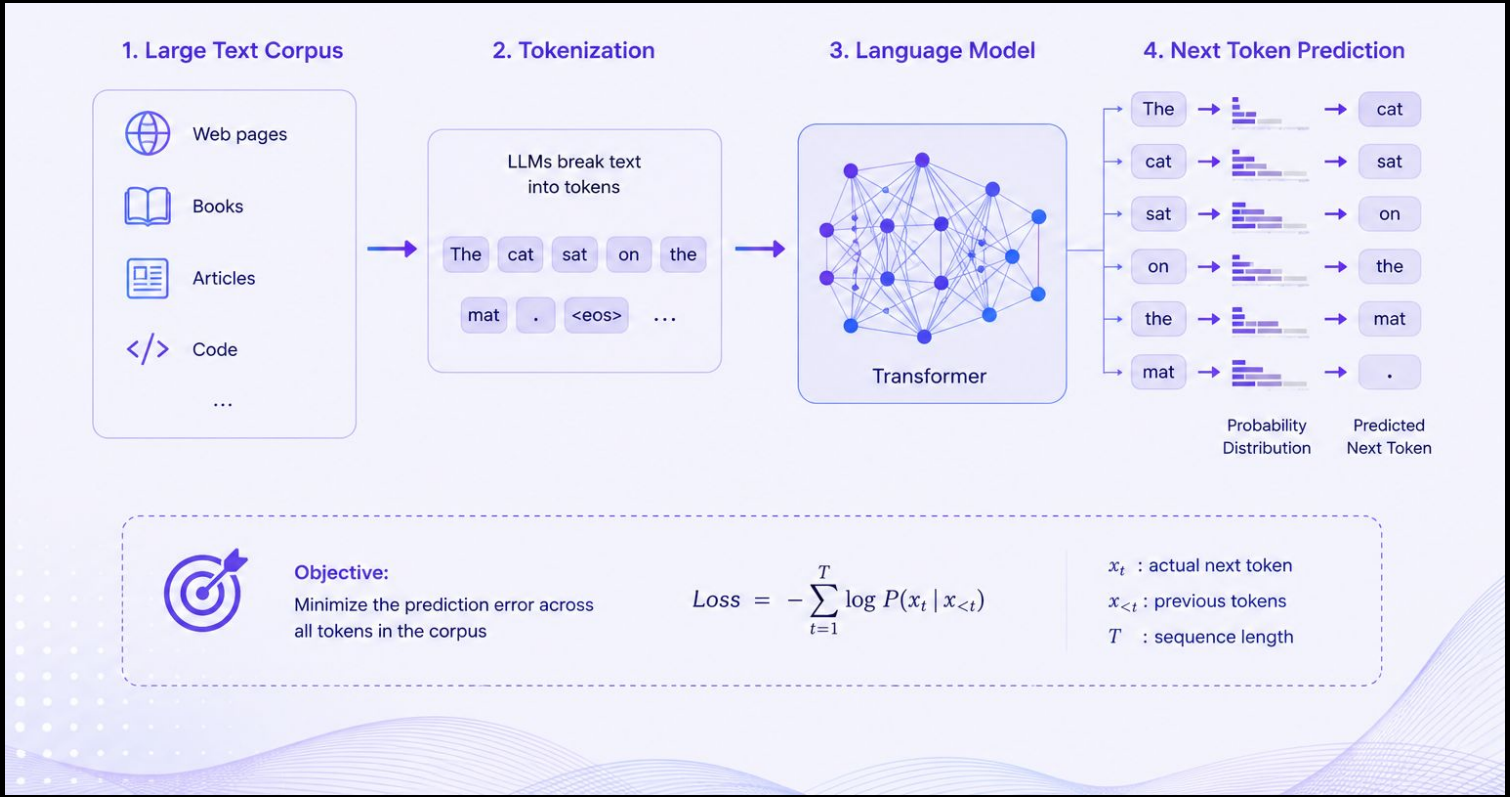
KL divergence

“Kullback-Leibler (KL) divergence is a mathematical measure from information theory that **calculates how one probability distribution differs from a reference distribution.** It essentially measures the "information loss" or "excess surprise" incurred when you use an approximating distribution to represent the true distribution of data.” (Wiki)

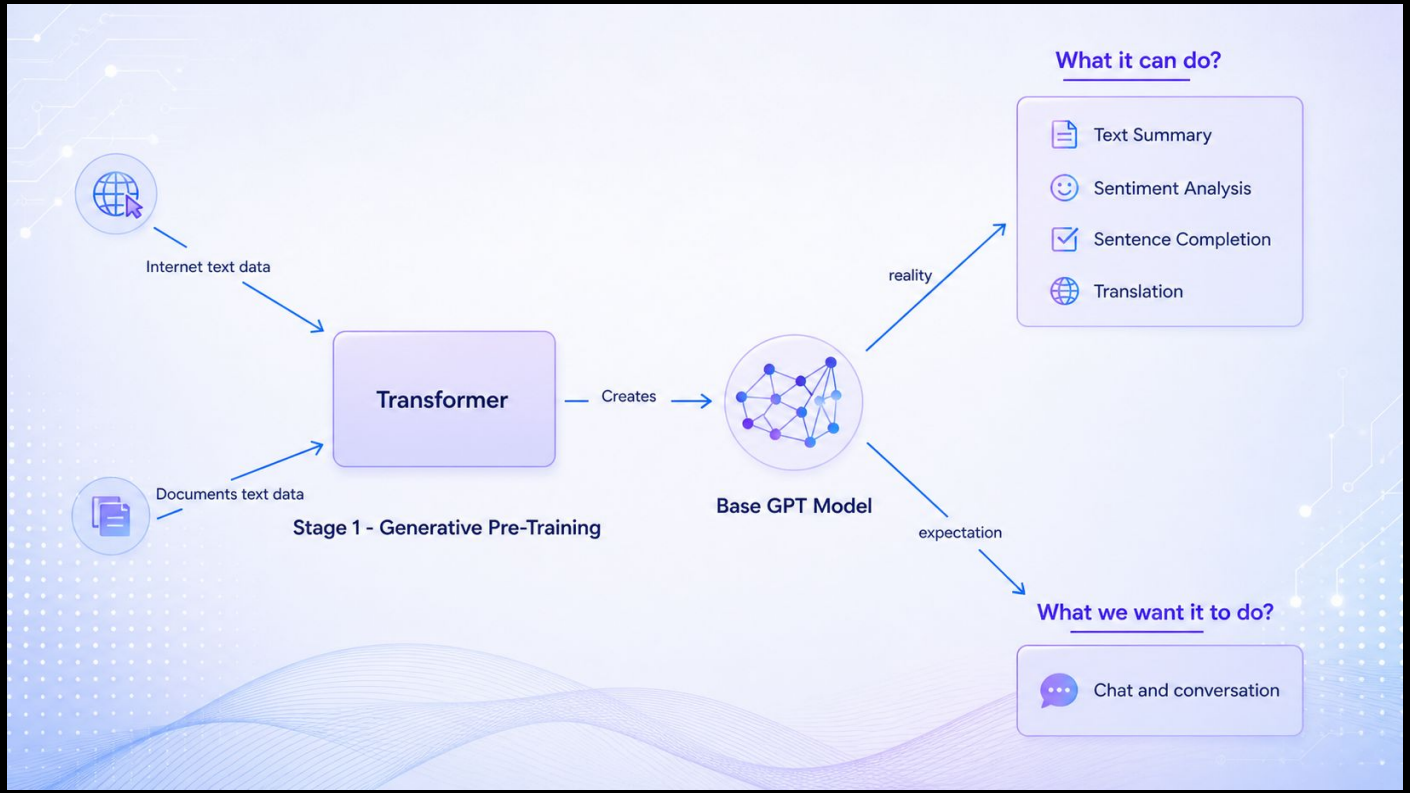
$$D_{\mathrm{KL}}(p||q) = \mathbb{E}\left[\log p(x) - \log q(x)\right] \quad \text{(Continuous Case)}$$

$$D_{\mathrm{KL}}(p||q) = \sum_{i=1}^{N} p(x_i) \cdot (\log p(x_i) - \log q(x_i)) \quad \text{(Discrete Case)}$$

Pre-Training



Pre-Training Outcome – Base Model



Pre-training

Prerequisites

- Plain text
  - Preferably diverse and “clean” in some sense
  - The more the merrier
- Computation power ($$)
  - The more the merrier
- Your favorite architecture
  - Not one correct answer

Pre-training

Data Example

The Boston Celtics again stand alone among NBA champions.
Tatum had 31 points, 11 assists and eight rebounds, and the Celtics topped the Dallas Mavericks 106-88 on Monday night to win the franchise's 18th championship, breaking a tie with the Los Angeles Lakers for the most in league history.
Boston earned its latest title on the 16th anniversary of hoisting its last Larry O'Brien Trophy in 2008. It marks the 13th championship won this century by one of the city's Big 4 professional sports franchises.
"It means the world," Tatum said on stage after the team received the trophy from NBA Commissioner Adam Silver. "It's been a long time. And damn I'm grateful."
Jaylen Brown added 21 points, eight rebounds and six assists, and was voted the NBA Finals MVP.
"I share this with my brothers and my partner in crime Jayson Tatum," Brown said after the 107th career playoff game he and Tatum have played together — the most for any duo before winning a title.
Jrue Holiday finished with 15 points and 11 rebounds. Center Kristaps Porzingis also provided an emotional lift, returning from a two-game absence because of a dislocated tendon in his left ankle to chip in five points in 17 minutes.

Pre-training

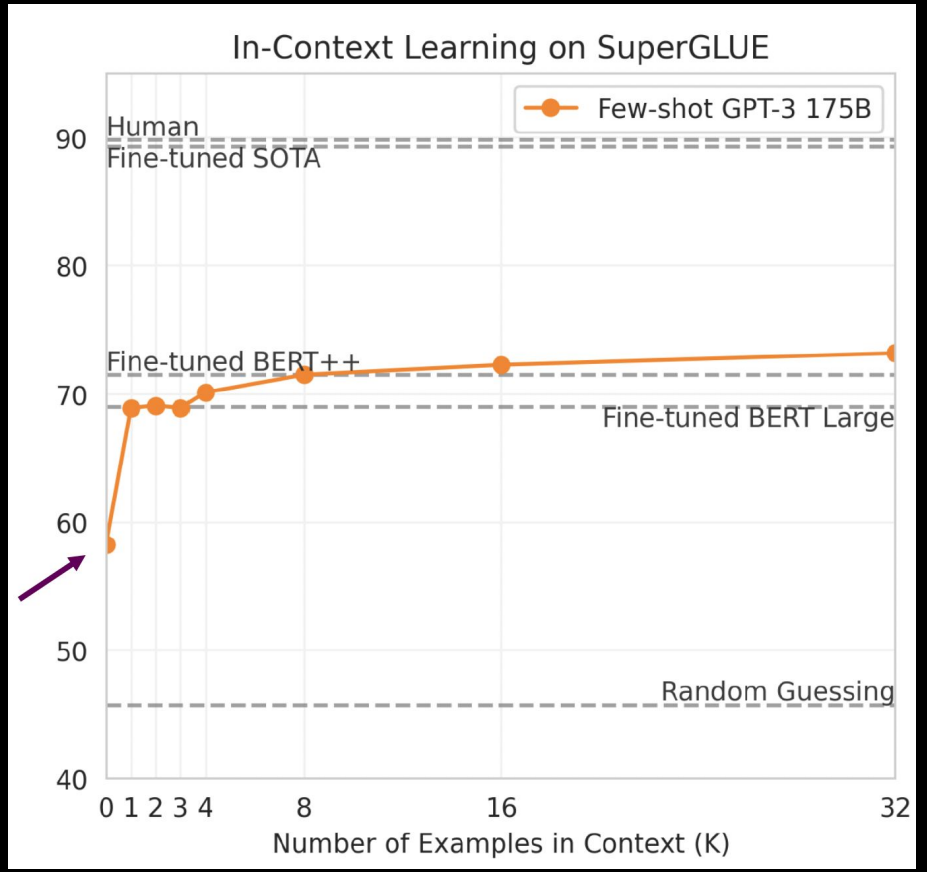
Things Pre-training mainly learns

- Syntax - "*I put ___ fork down on the table.*"
- Trivia - "*Stanford University is located in _____, California.*"
- Coreference - "*The woman walked across the street, checking for traffic over ___ shoulder.*"
- lexical semantics - "*I went to the ocean to see the fish, turtles, seals, and _____.*"
- Sentiment - "*Overall, the value I got from the two hours watching it was the sum total of the popcorn and the drink. The movie was ___.*"

Pre-training

Prompting is an enabler

- Few shots
- Chain of thought prompting



Pre-training

What's missing?

```
This is a tweet sentiment classifier
Tweet: "I loved the new Batman movie!"
Sentiment: Positive
###
Tweet: "I hate it when my phone battery dies."
Sentiment: Negative
###
Tweet: "My day has been 👍"
Sentiment: Positive
###
Tweet: "This is the link to the article"
Sentiment: Neutral
###
Tweet: "This new music video blew my mind"
Sentiment: Positive
```

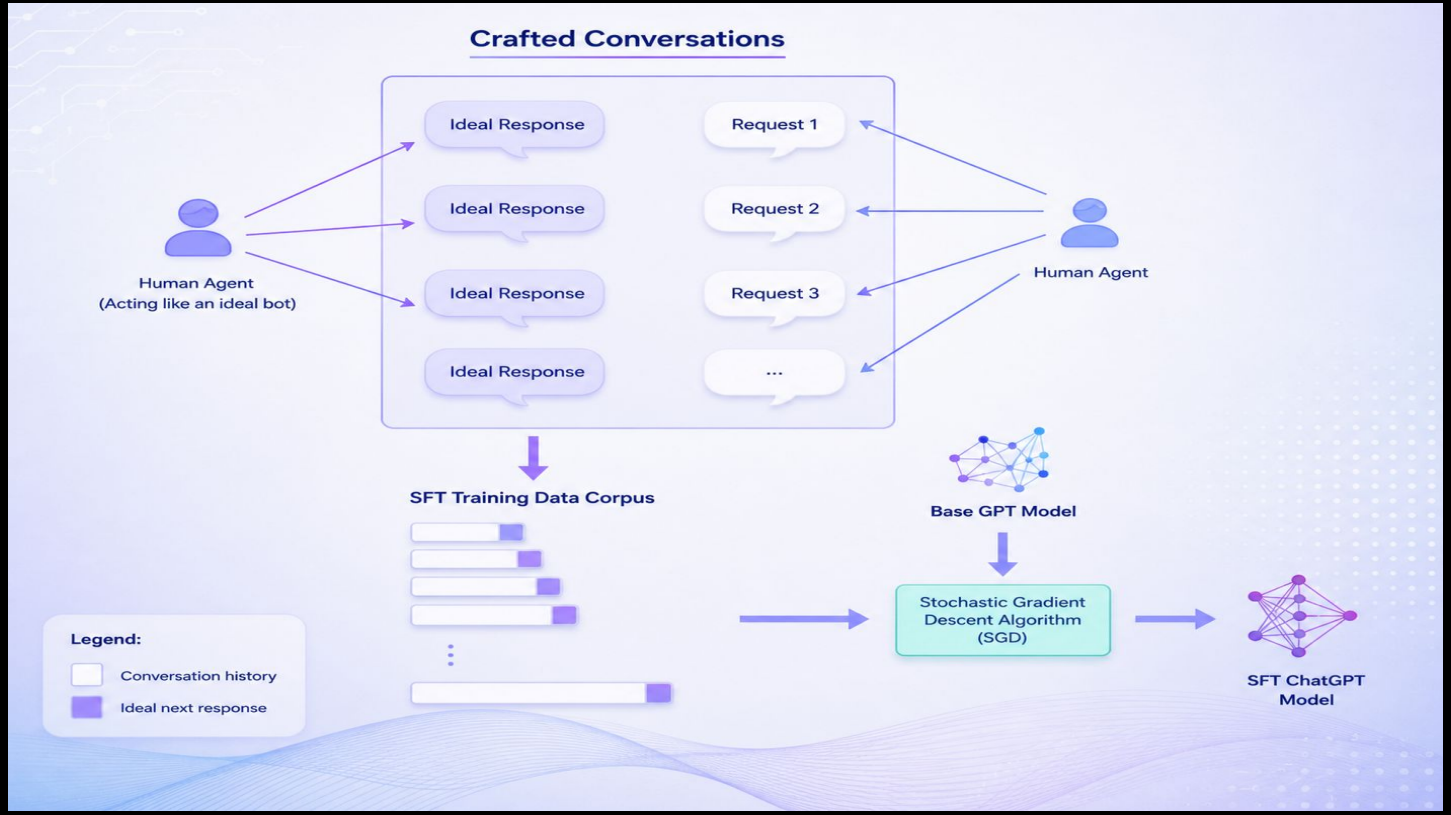
Pretraining Summary

- Teaches - “how to speak”, syntax, semantics, factual knowledge
- Data - plain-text
- Training task - next-token prediction
- Loss - cross-entropy
- Limitations - just text completion

Fine-tuning

"...OpenAI has hired contractors in multiple parts of the world including Latin America and Eastern Europe. About 600 … enriching AI training datasets with contextual information … The remaining **400** of the contractors are **programmers**, today's report elaborated. It's believed they will **write code** … **OpenAI is seeking not only code examples, but also natural language descriptions of the underlying thought process**.

Supervised Fine-Tuning (SFT)



Fine-tuning

Prerequisites

- Domain specific structured text
  - High quality - want code generation? Hire software developers ($$$)
  - Can contain several tasks
  - The more the merrier

Fine-tuning

Data Example

Instruction	Output
Answer the following question by reasoning step-by-step. A team lost its possession of the ball when the shot clock showed “15”, how long was the possession?	The shot clock starts at “24” when a team gains possession. The clock showed “15” therefore the possession lasted 24-15=9 seconds.
Answer the following question. When a shooter uses a pump fake to get the defender in the air at the wrong time, and the shooter leaps into the defender, who will get a foul call?	A foul is called on the defensive player.
Translate to French: Since a defender must leap to block a jumper, the shooter may use a pump fake to get the defender in the air at the wrong time and so have a clear shot. If the shooter leaps into the defender, a foul is called on the defensive player, whereas the shooter is awarded two or three free throws according to the value of a missed attempt, or a single free throw.	Puisqu'un défenseur doit sauter pour bloquer un sauteur, le tireur peut utiliser une feinte de pompe pour faire décoller le défenseur au mauvais moment et ainsi avoir un tir clair. Si le tireur saute sur le défenseur, une faute est sifflée sur le joueur défensif, tandis que le tireur se voit accorder deux ou trois lancers francs selon la valeur d'une tentative manquée, ou un seul lancer franc.

Fine-tuning

Limitations

- Write a recap to the NBA finals for children, narrated by Spiderman
  - Answer A: Hey there, little heroes! It's your friendly neighborhood Spider-Man here, swinging by to give you the inside scoop on the NBA Finals. My spidey senses are tingling just thinking about all the excitement on the court!...
  - Answer B: Hello kids, i'm spiderman, let me recap the NBA finals for you. The Celtics won 106-88, great performance of Tatum.

- Tasks like open-ended creative generation have no right answer.

Fine-tuning

Limitations

- Humans generate suboptimal answers - we want the model to do better.

- Language modeling penalizes all token-level mistakes equally, but some errors are worse than others.
  - What is the topic? "*Jaylen Brown seemed like you were a little bit more, well, I guess selective with your offense. So it was 15 shots tonight. What was it that you saw that made you shoot less and maybe do other things?*"
    - Brown's going all out on offense.
    - Interviewer bitching about 15 shots not being enough

Fine-tuning

Let's recap

- Teaches - use-case oriented completion style/type, factual knowledge
- Data - high-quality custom made text (e.g. question answering)
- Training task - next-token prediction
- Loss - cross-entropy
- Problems:
  - High quality FT data is expensive.
  - Open-ended creative generation has no right answer.
  - Some errors are worse than others.
  - We want "better than human" answers.

SFT vs. RL performance graph

From InstructGPT paper

It always depends on the downstream task

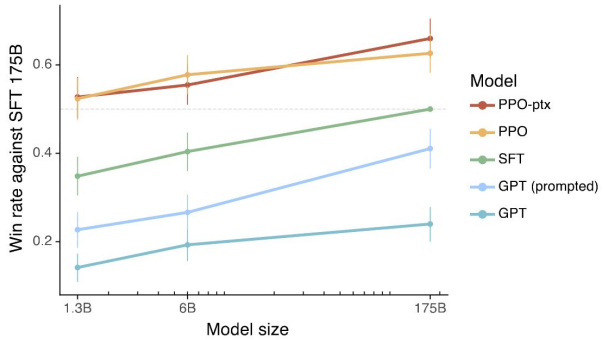


Figure 1: Human evaluations of various models on our API prompt distribution, evaluated by how often outputs from each model were preferred to those from the 175B SFT model. Our InstructGPT models (PPO-ptx) as well as its variant trained without pretraining mix (PPO) significantly outperform the GPT-3 baselines (GPT, GPT prompted); outputs from our 1.3B PPO-ptx model are preferred to those from the 175B GPT-3. Error bars throughout the paper are 95% confidence intervals.

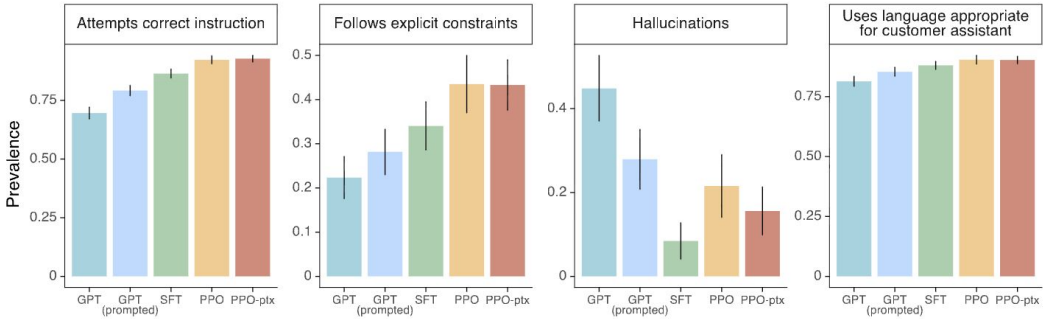
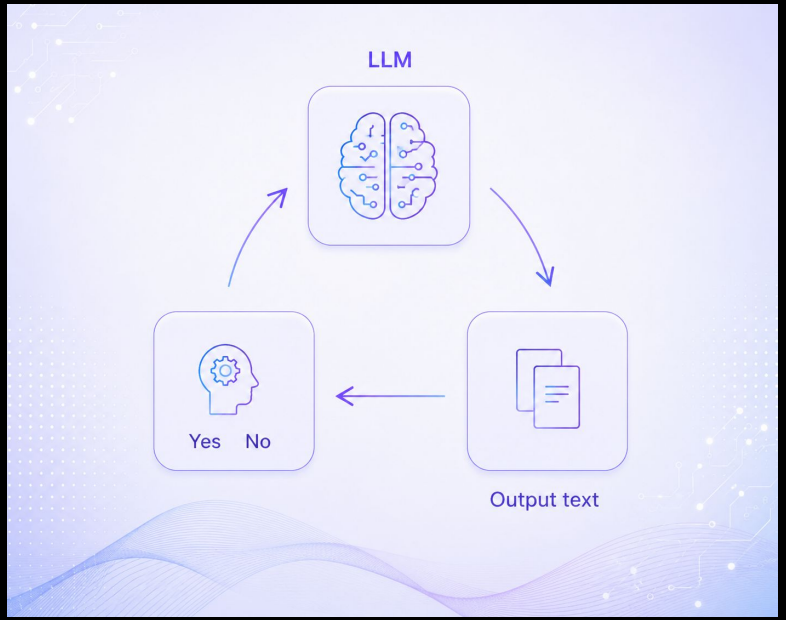


Figure 4: Metadata results on the API distribution. Note that, due to dataset sizes, these results are collapsed across model sizes. See Appendix E.2 for analysis that includes model size. Compared to GPT-3, the PPO models are more appropriate in the context of a customer assistant, are better at following explicit constraints in the instruction and attempting the correct instruction, and less likely to 'hallucinate' (meaning, making up information on closed domain tasks like summarization).

Any Questions?

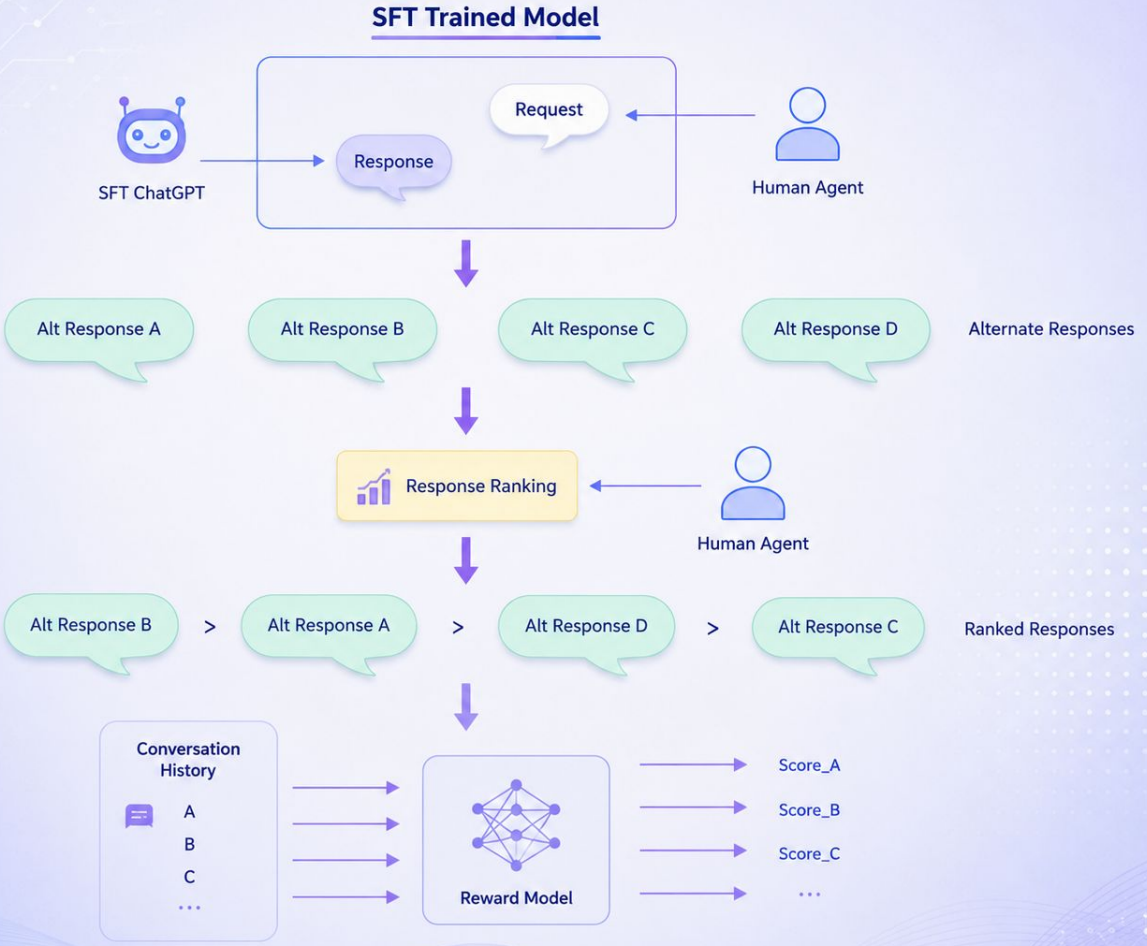


Human Preference Alignment (RLHF)



Reinforcement Learning through Human Feedback (RLHF)

→ Reward Model



Collect comparison data, and train a reward model.

Train a reward model (RM) .

The model predicts a single score for each pair of prompt and response.

*RM model starts with SFT, removes the output linear+softmax layer at the output, and attaches a new linear layer to predict a scalar (**instead of probability over vocabulary**). The RM training data is a dataset of comparisons between model outputs, where labelers indicate which output they prefer for a given input. RM dataset has*

33k training prompts.

Collect comparison data, and train a reward model.

RM is effectively a learning-to-rank model to correctly predict the rank order over K responses using the following pairwise ranking loss:

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x,y_w,y_l)\sim D}[log(\sigma(r_\theta(x,y_w) - r_\theta(x,y_l)))]$$

Where $r_\theta(x, y_w)$ is the scalar output of the reward model for prompt x and completion y with parameters θ, y_w is the preferred completion out of the pair of y_w and y_l, and D is the dataset of human comparisons.

Optimize a policy against the reward model using PPO.

Now we have a fine-tuned model $\pi_{SFT}(y|x)$ *and a reward model* $r_\theta(x, y)$.

We will use the reward model to find a more optimal parameter set ϕ *for the model through a reinforcement learning (RL) set up:*

- *The environment is a bandit environment that presents a random customer prompt and expects a response to the prompt.*
- *Given the prompt and response, it produces a reward determined by the reward model and ends the episode.*

Optimize a policy against the reward model using PPO.

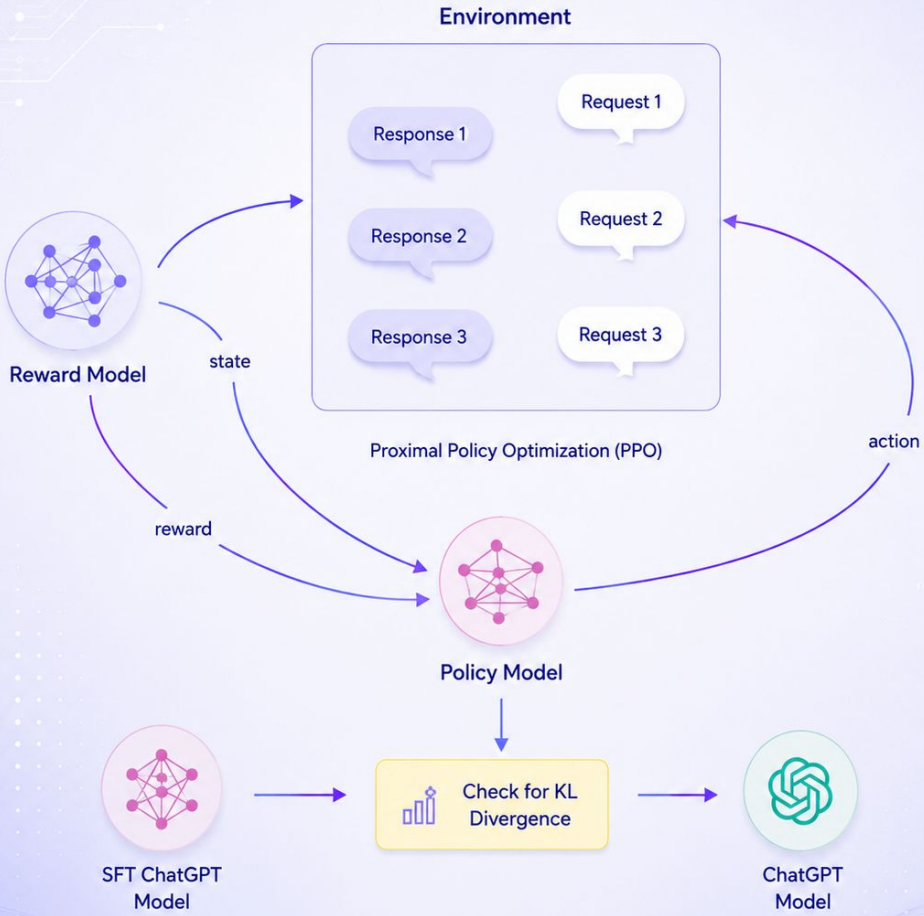
We maximize the following combined objective function in RL training.

The policy gradient objective is:

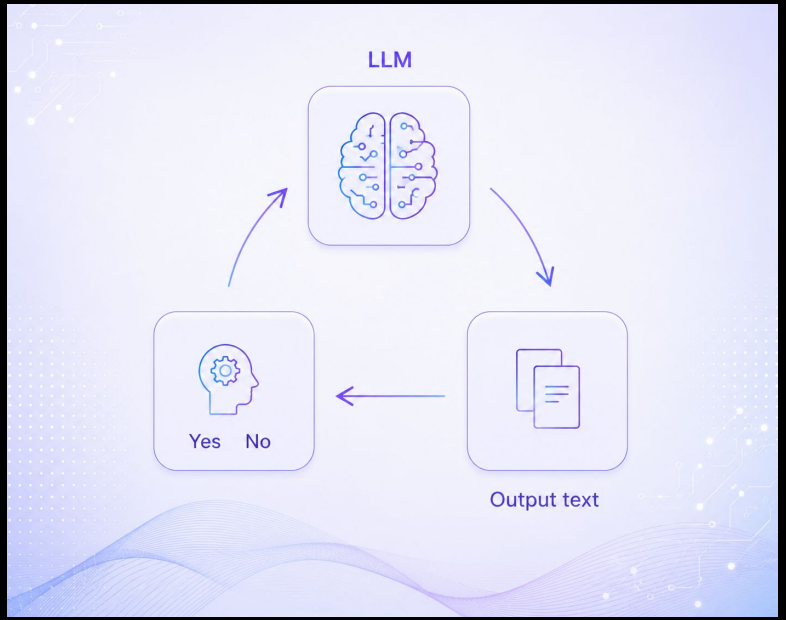
$$\text{objective}(\phi) = E_{(x,y)\sim D_{\pi_\phi^{RL}}}\left[r_\theta(x,y) - \beta\log\left(\frac{\pi_\phi^{RL}(y|x)}{\pi^{SFT}(y|x)}\right)\right] + \gamma E_{x\sim D_{pretrain}}\left[\log(\pi_\phi^{RL}(x))\right]$$

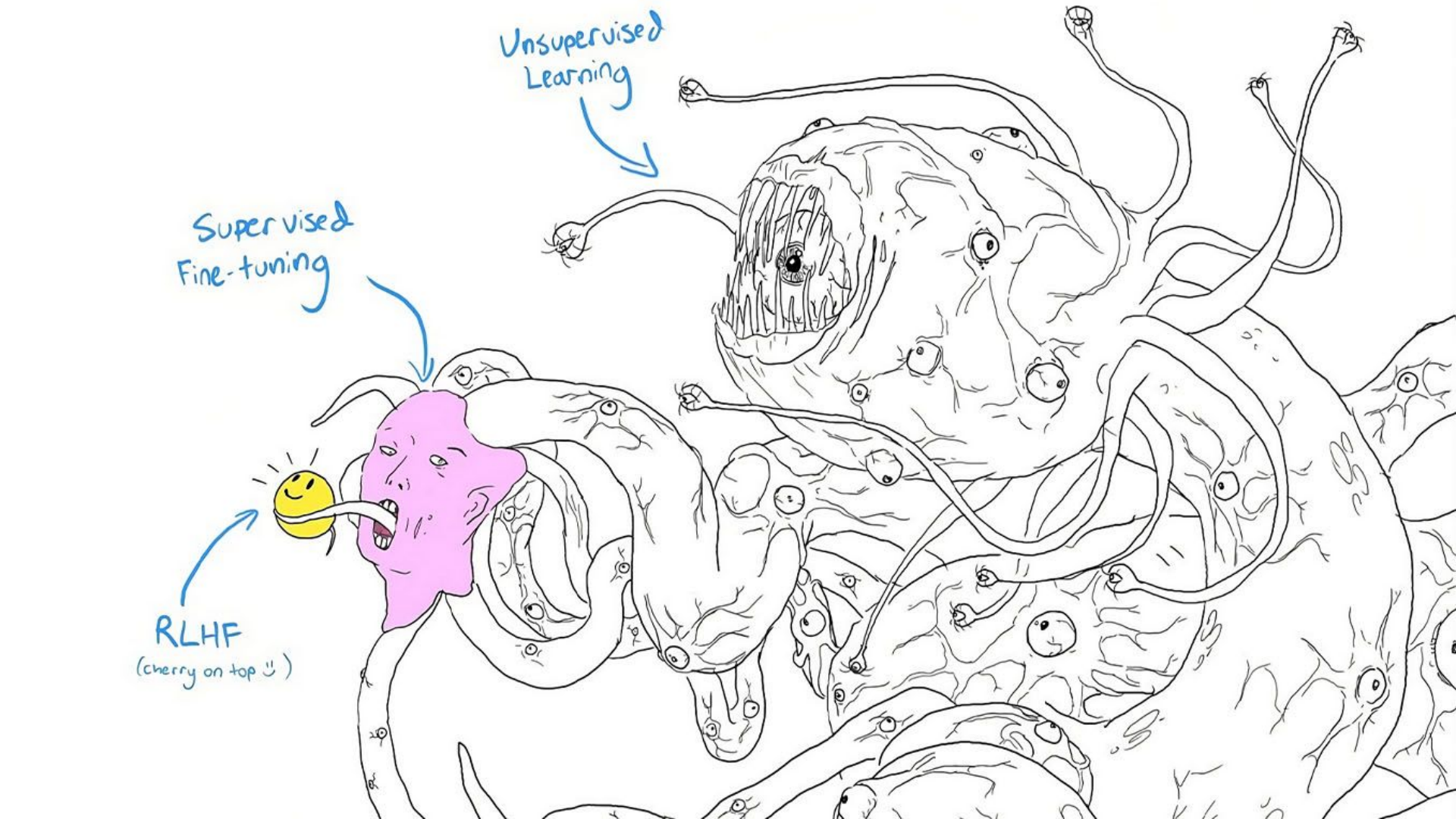
- The **red** portion of the objective is about maximizing reward.
- The portion is the KL divergence between $\pi_\phi^{RL}(y|x), \pi^{SFT}(y|x)$, minimizing this KL term keeps the learned RL mode close to SFT and avoids overfitting of the reward model.
- The **green** portion is an added loss term to ensure the RL model keeps up its performance in the original language modeling objective - empirically, this helps to $\pi_\phi^{RL}(y|x)$ continue perform well in common NLP tasks.

PPO Setup


Environment
Request 1
Response 1
Request 2
Response 2
Request 3
Response 3
Proximal Policy Optimization (PPO)
Reward Model
state
action
reward
Policy Model
SFT ChatGPT Model
Check for KL Divergence
ChatGPT Model

Optimizing for Human Preference




Unsupervised Learning
Supervised Fine-tuning
RLHF
(cherry on top ᵕ̈)

Preference Data

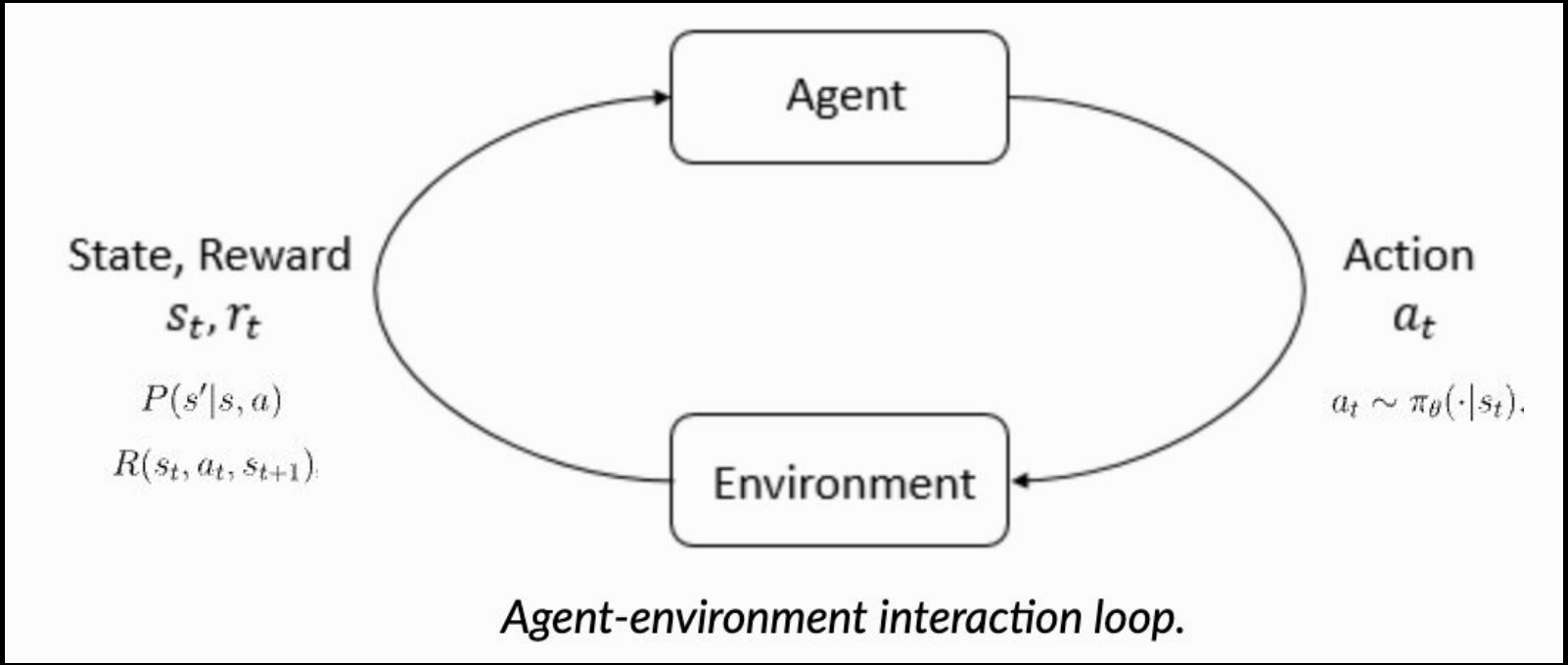
- Write a recap to the NBA finals for children, narrated by Spiderman
  - Answer A: *Hey there, little heroes! It's your friendly neighborhood Spider-Man here, swinging by to give you the inside scoop on the NBA Finals. My spider senses are tingling just thinking about all the excitement on the court!...*
  - Answer B: *Hello kids, i'm spiderman, let me recap the NBA finals for you. The Celtics won 106-88, great performance of Tatum.*
- Label: A better than B
  - Multiple labelers, multiple answers
  - Much cheaper data - it's easier to verify than to solve (until P=NP proves otherwise)

RL Recap



RL Intro

Interaction loop



Agent-environment interaction loop.

RL Recap

In General - Why can't RL be framed as supervised learning (SL)?

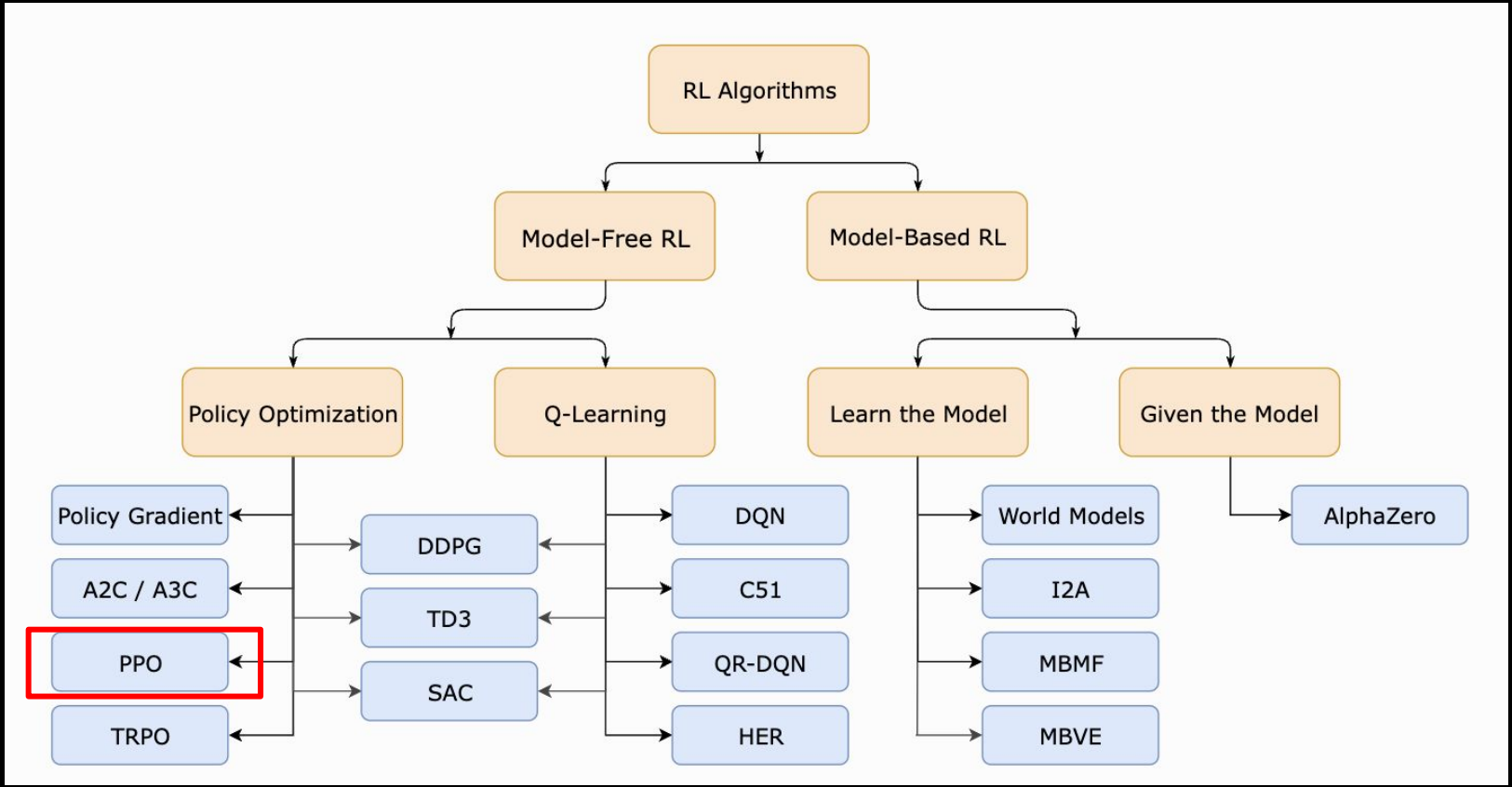
- The objective is over trajectories, not pointwise predictions
- Samples are not i.i.d.

  SL assumes data is drawn i.i.d. from a fixed distribution. RL violates this in two compounding ways:

  - **Policy-dependent distribution**: the data you collect depends on your current policy. As the policy updates, the distribution of states and actions shifts – so you're never training on a stable dataset. This is the core issue behind on-policy vs. off-policy methods.
  - **Delayed feedback**: in SL, the label for x_i is immediately available. In RL, the consequence of an action at step t may only manifest as reward at step t+k. The model must learn to credit past actions for future outcomes – the **credit assignment problem** – which has no analog in standard SL.
- Exploration-Exploitation
  - In SL, the training set is fixed and given. In RL, the agent must actively generate its own training data by taking actions

RL Intro

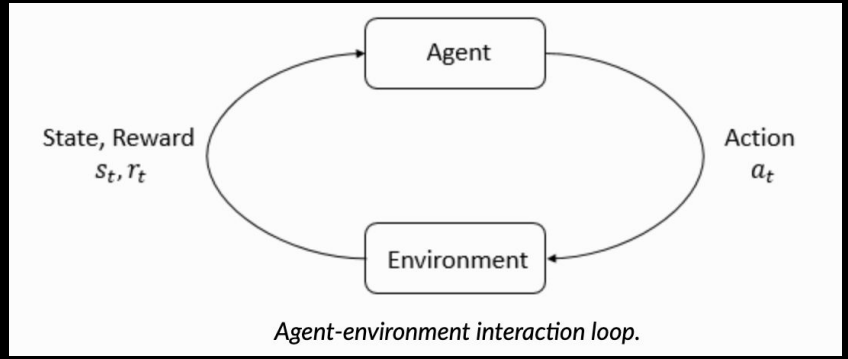
Taxonomy



LM as RL

Modeling LM as a RL problem

- State - Context
- Agent/Policy - LLM
- Action - Completion (token)
- Environment - Prompts
- Reward - human preference



Agent-environment interaction loop.

- Example:

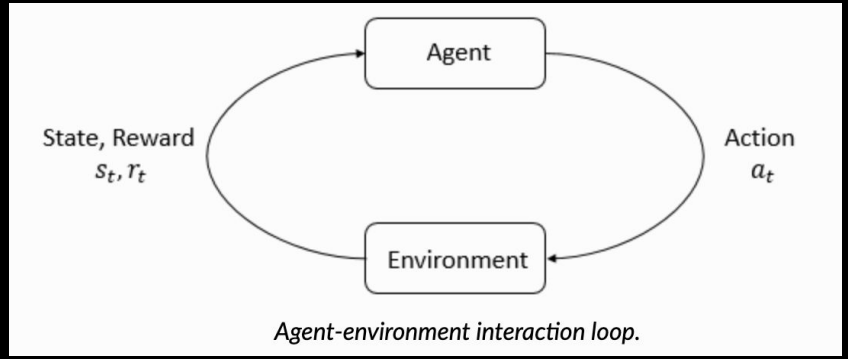
  Solve 2x-6=x. Sure, first subtract x from both

  -0.1 -0.2 -0.3 -0.2 -0.6 -0.03 -0.11 -0.2 -0.1

LM as RL

Who's behind the mask?

- State - Context
- Agent/Policy - LLM
- Action - Completion (token)
- Environment - Prompts
- Reward - human preference



Agent-environment interaction loop.

- Problems:
  - Per token human preference?
  - Generalization? (preference is collected before-hand)

The Reward Model – Summary

BLOCK 2 · RLHF

Architecture

- Start from the SFT checkpoint (same tokenizer, same weights)
- Replace the language model head with a linear layer → scalar
- Input: [prompt + response]
- Output: single reward score $r \in \mathbb{R}$
- Train with Bradley-Terry ranking loss on preference pairs

Reward hacking

The policy will find gaps in the reward model.
If RM rewards verbosity, you get long-winded answers.
If RM rewards confident tone, you get hallucinations.
KL penalty limits but doesn't eliminate this.

KL penalty — why it's critical

RL objective = $E[r(x,y)] - \beta \cdot KL[\pi \,||\, \pi_{SFT}]$

β controls the trust region. Too small → hacking.
Too large → no learning. $\beta \approx 0.01–0.1$ in practice.

Any Questions?



Proximal Policy Optimization (PPO)

Success Stories



OpenAI Five (Gaming)

“Our team of five neural networks, OpenAI Five, has started to defeat amateur human teams at Dota 2.”



The project

Solving Rubik's Cube with a robot hand (OpenAI)



The project

PPO

Proximal Policy Optimization (PPO) (Schulman et al., 2017) is an actor-critic RL algorithm that is widely used in the RL fine-tuning stage of LLMs (Ouyang et al., 2022). In particular, it optimizes LLMs by maximizing the “clipped surrogate objective”.

Goal:
Maximize reward while staying close to a pre-trained base model.

Overview PPO

PPO helps align language models to human preferences using feedback, while ensuring the model doesn't drift too far from its base behavior.

Key components:

- State $s_t=(x,y_{<t})$ – prompt + tokens so far
- Action $a_t=y_t$ next token
- Policy $\pi_\theta(a_t|s_t) = \pi_\theta(y_t \mid x,y_{<t})$
- Trajectory = one full generation $\boldsymbol{\tau} = (x, y_{1:T})$
- Reward is **terminal and sparse** : $r_{RM}(x,y)$ arrives only at the EOS token

The objective and the gradient we can't compute

We want $\nabla_\theta J$ but τ's distribution depends on θ – can't just differentiate samples.

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[R(\tau)\right], \qquad R(\tau) = \sum_t r_t$$

We want to derive J, but the thing we're sampling from "moves" as we change θ. We can't just differentiate samples – we need a gradient we can estimate from rollouts.

The log-derivative trick (REINFORCE)

$$\nabla_\theta J(\theta) = \int \nabla_\theta \pi_\theta(\tau)\, R(\tau)\, d\tau = \int \pi_\theta(\tau)\, \underbrace{\nabla_\theta \log \pi_\theta(\tau)}_{\text{log-deriv identity}} R(\tau)\, d\tau$$

$$\boxed{\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta}$$

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\, R(\tau)\, \nabla_\theta \log \pi_\theta(\tau) \,\right]}$$

From trajectory to per-token gradient

Dynamics terms drop out because they don't depend on θ:

$$\log \pi_\theta(\tau) = \sum_t \log \pi_\theta(a_t \mid s_t) \; + \; \underbrace{\text{const}(\theta)}_{\text{env dynamics}}$$

$$\nabla_\theta J(\theta) = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau) \Big]$$

From trajectory to per-token gradient

The trajectory distribution is not just the policy product – the environment is in it:

$$p_\theta(\tau) = \rho_0(s_0) \prod_t \pi_\theta(a_t \mid s_t)\, P(s_{t+1} \mid s_t, a_t)$$

$$\log p_\theta(\tau) = \sum_t \log \pi_\theta(a_t \mid s_t) \; + \; \underbrace{C(\tau)}_{\substack{\text{initial state + dynamics} \\ \nabla_\theta C(\tau) = 0}}$$

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau) \Big]$$

The policy gradient (REINFORCE)

The log-derivative trick gives an estimator we can sample:

$$\nabla_\theta J = \mathbb{E}_{\tau \sim \pi_\theta} \left[R(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right]$$

What we got?
push up the log-prob of tokens in good trajectories, push down the bad ones.

Causality → reward-to-go

An action at t can't affect past rewards, so replace R(τ) with reward-to-go:

$$\hat{R}_t = \sum_{t' \geq t} r_{t'} \qquad \Rightarrow \qquad \nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{R}_t \Big]$$

Subtract any action-independent $b(s_t)$:

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, (\hat{R}_t - b(s_t)) \Big]$$

Making it usable: reward-to-go, baseline, advantage

Raw REINFORCE works but is impossibly noisy. Three standard fixes, stated as results:

- *Reward-to-go – a token is only credited for rewards that come after it.*
- *Baseline – subtract an expected value $b(s_t)$, reduces variance, stays unbiased.*
- *The best baseline is the value function $V^\pi(s_t)$ which turns reward-to-go into the advantage:*

$$V^\pi(s_t) = \mathbb{E}_\pi[\hat{R}_t \mid s_t], \quad Q^\pi(s_t, a_t) = \mathbb{E}_\pi[\hat{R}_t \mid s_t, a_t]$$

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$$

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A^\pi(s_t, a_t)\Big]$$

Any Questions?



Was this token better or worse than expected from this state?

Every *algorithm ahead: PPO, TRPO, GRPO – is "just" a different way to estimate A*

On-Policy policy gradient problem

The expectation is taken over trajectories drawn from π_θ– the current policy. That subscript is the whole story. The formula is only a valid estimate of the gradient at the specific θ that generated the data.

The moment you apply one gradient step, θ→θ, your policy is now $\pi_{\theta'}$ and the data you collected was sampled from π_θ the "old" one. The expectation you'd need is now over $\pi_{\theta'}$ which you don't have samples from.

$$\nabla_\theta J = \mathbb{E}_{\tau \sim \pi_\theta}\left[\cdots \right]$$

The general identity

*You can compute an expectation under one distribution **p** using samples from another distribution **q** as long as you reweight each sample by the probability ratio:*

$$\mathbb{E}_{x\sim p}[f(x)] = \mathbb{E}_{x\sim q}\left[\frac{p(x)}{q(x)} f(x)\right]$$

Apply this with p =π_θ (the policy you want the gradient for) and q=$\pi_{\theta old}$ (the policy that actually generated your batch). The reweighting factor is exactly the ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

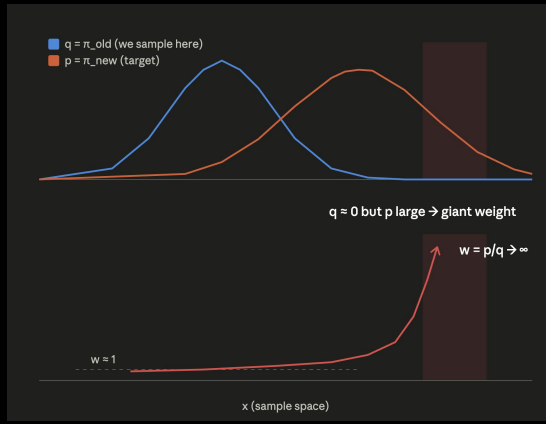
$$L(\theta) = \mathbb{E}_{t,\, \tau\sim\pi_{\theta_{\text{old}}}}\left[\rho_t(\theta)\, \hat{A}_t\right]$$

Why p(x) should be closed to q(x)?

The gradient estimate becomes unreliable!
Why? If one sample has a huge p/q, that single term dominates the whole sum - ***noisy estimation.***

That noise causes divergence:

1. *A high-variance batch produces a bad, erratic gradient step – the policy lurches in some arbitrary direction.*
2. *That step moves π_θ further from $\pi_{\theta old}$, not closer.*
3. *Farther apart → the ratios p/q get even more extreme on the next batch → variance grows again.*
4. *And critically, the policy generates the next batch of data. A corrupted policy produces corrupted rollouts, which produce a worse gradient, which corrupts the policy more.*



The Surrogate Objective - TRPO

TRPO – making "close enough" precise



That is expensive and NOT “user friendly” at LLM scale

Maximize the surrogate, but never step outside a KL ball of radius δ.

PPO clipped surrogate

*PPO replaces the hard KL constraint with a **clip** that simply stops rewarding the optimizer once the ratio leaves a band*

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)}\hat{A}_t,\ \mathrm{clip}\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right]$$

PPO Objective Function

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})}, 1-\varepsilon, 1+\varepsilon \right) A_t \right]$$

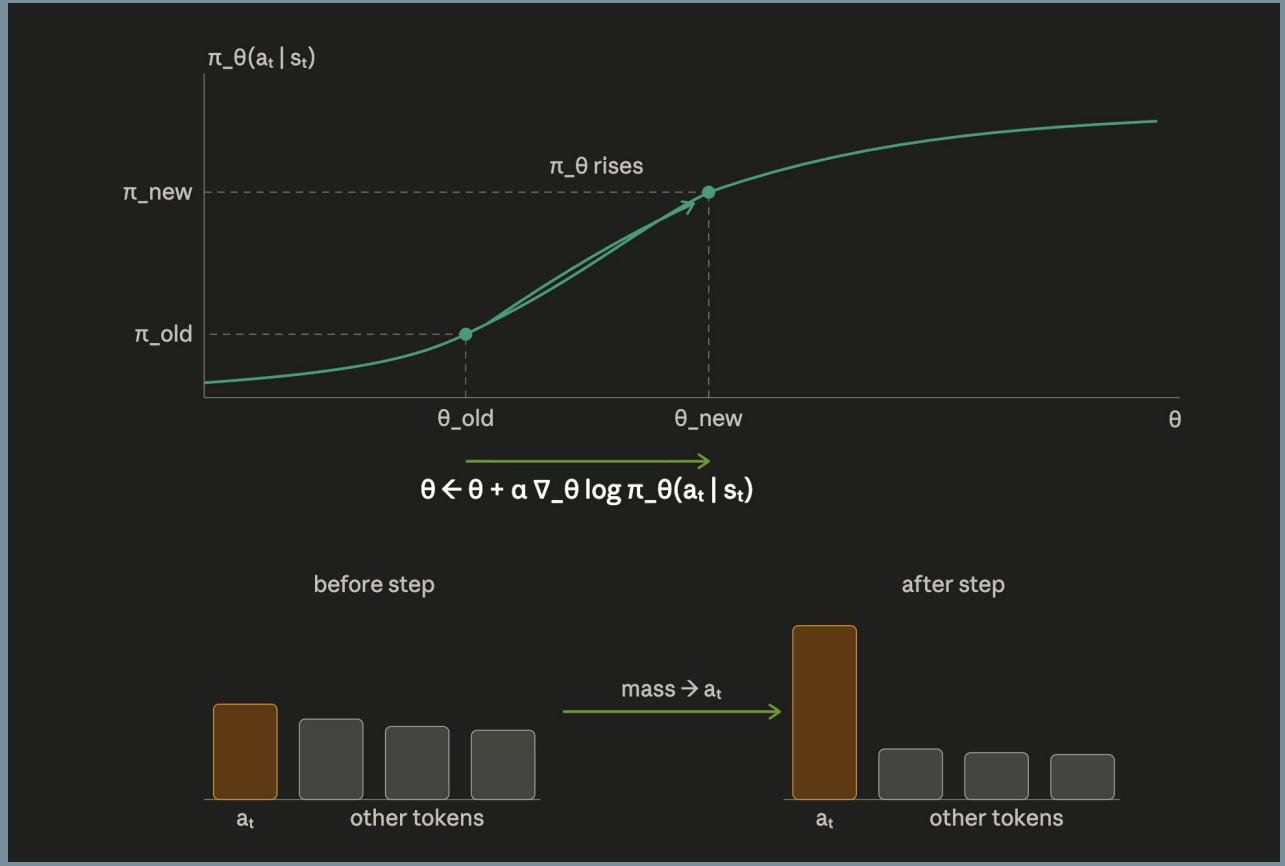
Where:

$\left[\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})} \right]$ ⟶ This measures how much the policy diverges from the old model at that specific step.

A_t ⟶ The advantage term is crucial to guide learning based on better-than-expected outcomes.

$1-\varepsilon, 1+\varepsilon$ ⟶ The clip range

Optimization Step

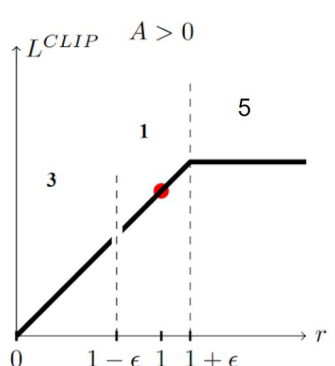


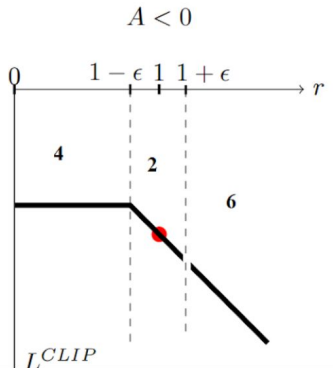
PPO

Stability - clipped loss

$$L(s, a, \theta_k, \theta) = \min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s,a),\ \ \text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, 1-\epsilon, 1+\epsilon\right) A^{\pi_{\theta_k}}(s,a)\right)$$

	$p_t(\theta) > 0$	A_t	Return Value of *min*	Objective is Clipped	Sign of Objective	Gradient
1	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	$+$	$p_t(\theta)A_t$	no	$+$	✓
2	$p_t(\theta) \in [1-\epsilon, 1+\epsilon]$	$-$	$p_t(\theta)A_t$	no	$-$	✓
3	$p_t(\theta) < 1-\epsilon$	$+$	$p_t(\theta)A_t$	no	$+$	✓
4	$p_t(\theta) < 1-\epsilon$	$-$	$(1-\epsilon)A_t$	yes	$-$	**0**
5	$p_t(\theta) > 1+\epsilon$	$+$	$(1+\epsilon)A_t$	yes	$+$	**0**
6	$p_t(\theta) > 1+\epsilon$	$-$	$p_t(\theta)A_t$	no	$-$	✓





Any Questions?



Now how can we estimate the Advantage Function?

GAE *Generalized Advantage Estimation*

What Is GAE (Generalized Advantage Estimation)?

GAE improves learning stability by combining rewards across time steps while still allowing bootstrapping from the value model.

- Reduces **variance** while preserving low **bias**
- Combines short-term and long-term rewards
- γ: discount factor, λ: GAE smoothing factor

Temporal Difference residual: $$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Reward I actually got, plus what I think the future is worth from where I landed, minus what I expected from where I started.

The reward structure in LLM PPO

In standard RLHF PPO (InstructGPT-style), the per-token reward is:

$$r_t = \underbrace{-\beta \log \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{ref}}(a_t \mid s_t)}}_{\text{per-token KL — dense}} + \underbrace{r_{\text{RM}} \cdot 1[t = T]}_{\text{sequence score — sparse}}$$

Here a token is an action, the state is prompt + tokens-so-far.
The reward-model score r_{RM} lands only at the final token, but the KL penalty is applied at every token. So it's not quite "reward only at the end".

A whole spectrum of estimators

We can estimate the advantage by looking ahead **n** steps before falling back on the value function. These can all be written as sums of TD (Temporal Difference) residuals:

$$\hat{A}_t^{(1)} = \delta_t$$

$$\hat{A}_t^{(2)} = \delta_t + \gamma\,\delta_{t+1}$$

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} (\gamma)^l\,\delta_{t+l}$$

Intuition → The two ends of the spectrum:

- Few steps (small **n**): lean heavily on **V**.
  Low variance, but biased – you inherit all of **V**'s errors.
- Many steps (large **n**, → Monte Carlo): lean on actual rewards.
  Low bias, but high variance – you inherit all the noise of a full trajectory.

Exponentially Weighted Average

GAE takes an exponentially weighted average of **all the n-step estimators** , with weight λ.
The messy weighted sum collapses into something strikingly simple:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}$$

$\lambda \in [0, 1]$ is the bias-variance dial:

$$\lambda = 0: \quad \hat{A}_t = \delta_t \qquad \text{(one-step, low variance / high bias)}$$

$$\lambda = 1: \quad \hat{A}_t = \sum_{l \geq 0} \gamma^l \delta_{t+l} \qquad \text{(Monte Carlo, low bias / high variance)}$$

Intermediate λ smoothly interpolates. In practice λ≈0.95 is a common sweet spot – mostly trusting observed rewards while damping the long-horizon noise.

Common Practice

The common setting in LLM RLHF is $\gamma = 1$ and $\lambda = 0.95$,
though $\lambda = 1$ is also frequently used.

$\gamma = 1$ is close to universal.
Within a single response there's **no principled reason** to time-discount – a good token early and a good token late should both get full credit for the final outcome. Setting $\gamma < 1$ would arbitrarily penalize longer responses by shrinking the credit that reaches early tokens. So almost everyone uses $\gamma = 1$. The original InstructGPT/summarization work from OpenAI used $\gamma = 1$.

From The Paper

3 ADVANTAGE FUNCTION ESTIMATION

This section will be concerned with producing an accurate estimate $\hat{A}_t$ of the discounted advantage function $A^{\pi,\gamma}(s_t, a_t)$, which will then be used to construct a policy gradient estimator of the following form:

$$\hat{g} = \frac{1}{N} \sum_{n=1}^{N} \sum_{t=0}^{\infty} \hat{A}_t^n \nabla_\theta \log \pi_\theta(a_t^n \mid s_t^n) \tag{9}$$

where n indexes over a batch of episodes.

Let V be an approximate value function. Define $\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$, i.e., the TD residual of V with discount γ (Sutton & Barto, 1998). Note that δ_t^V can be considered as an estimate of the advantage of the action a_t. In fact, if we have the correct value function $V = V^{\pi,\gamma}$, then it is a γ-just advantage estimator, and in fact, an unbiased estimator of $A^{\pi,\gamma}$:

$$\begin{aligned} \mathbb{E}_{s_{t+1}} \left[\delta_t^{V^{\pi,\gamma}} \right] &= \mathbb{E}_{s_{t+1}} \left[r_t + \gamma V^{\pi,\gamma}(s_{t+1}) - V^{\pi,\gamma}(s_t) \right] \\ &= \mathbb{E}_{s_{t+1}} \left[Q^{\pi,\gamma}(s_t, a_t) - V^{\pi,\gamma}(s_t) \right] = A^{\pi,\gamma}(s_t, a_t). \end{aligned} \tag{10}$$

However, this estimator is only γ-just for $V = V^{\pi,\gamma}$, otherwise it will yield biased policy gradient estimates.

Next, let us consider taking the sum of k of these δ terms, which we will denote by $\hat{A}_t^{(k)}$

$$\hat{A}_t^{(1)} := \delta_t^V \qquad = -V(s_t) + r_t + \gamma V(s_{t+1}) \tag{11}$$

$$\hat{A}_t^{(2)} := \delta_t^V + \gamma \delta_{t+1}^V \qquad = -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) \tag{12}$$

$$\hat{A}_t^{(3)} := \delta_t^V + \gamma \delta_{t+1}^V + \gamma^2 \delta_{t+2}^V = -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3}) \tag{13}$$

$$\hat{A}_t^{(k)} := \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k}) \tag{14}$$

The form that actually gets computed

In code you never write a closed form – you run the recursion ($\gamma = 1$):

$$\hat{A}_t = \delta_t + \lambda\,\hat{A}_{t+1}, \qquad \delta_t = -\beta\,\mathrm{kl}_t + V(s_{t+1}) - V(s_t) \;\; (+\,r_{\mathrm{RM}} \text{ at } t = T)$$

At $\lambda = 1$, $\gamma = 1$ the advantage is the full return-to-go minus the baseline.
Telescoping the value terms ($V(s_{T+1}) = 0$):

$$\hat{A}_t = r_{\mathrm{RM}} \; - \; \beta \sum_{l=t}^{T} \mathrm{kl}_l \; - \; V(s_t)$$

Value Model Loss

The value model helps estimate expected rewards to stabilize policy updates by computing advantages.

- $\left(\sum_{t=0}^{T} \gamma^t r_t \,\middle|\, s_0 = s\right)\Big\|_2^2$ is the the actual returns received.
- Trains the value head to predict expected future reward.

$$L_{\text{VF}} = \frac{1}{2}\left\| V_\theta(s) - \left(\sum_{t=0}^{T} \gamma^t r_t \,\middle|\, s_0 = s\right)\right\|_2^2$$

Why Token-Level Reward Matters

- More granular feedback
- Better credit assignment
- Richer signal for GAE and PPO
- Supports heuristic or learned rewards at finer scale

Instead of giving a single score for an entire response, token-level shaping helps the model learn which specific words or phrases were helpful, harmful, or neutral.

So the total objective is:

$$L = L^{\text{policy}} - c_1 L^{\text{value}} + c_2 L^{\text{entropy}}$$

Summary

- PPO is the backbone of RLHF for LLMs.
- GAE improves advantage estimation stability.
- Token-level reward shaping enables better control and alignment.
- Combine heuristics, learned rewards, and KL regularization for best results.

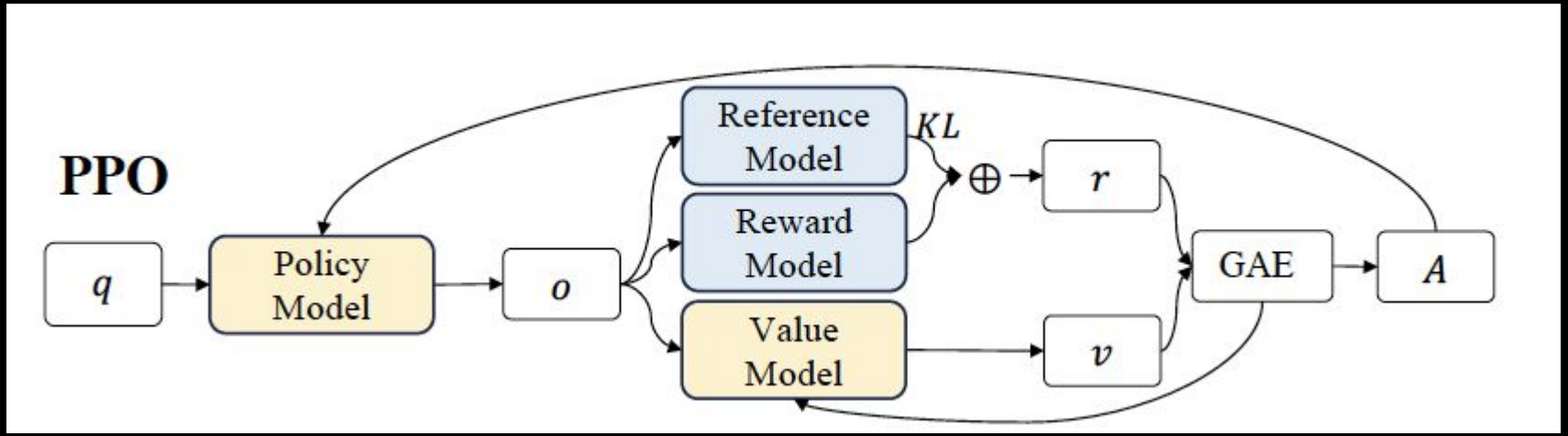
Token-level PPO with GAE is one of the most effective strategies for aligning large language models to nuanced human preferences.

PPO

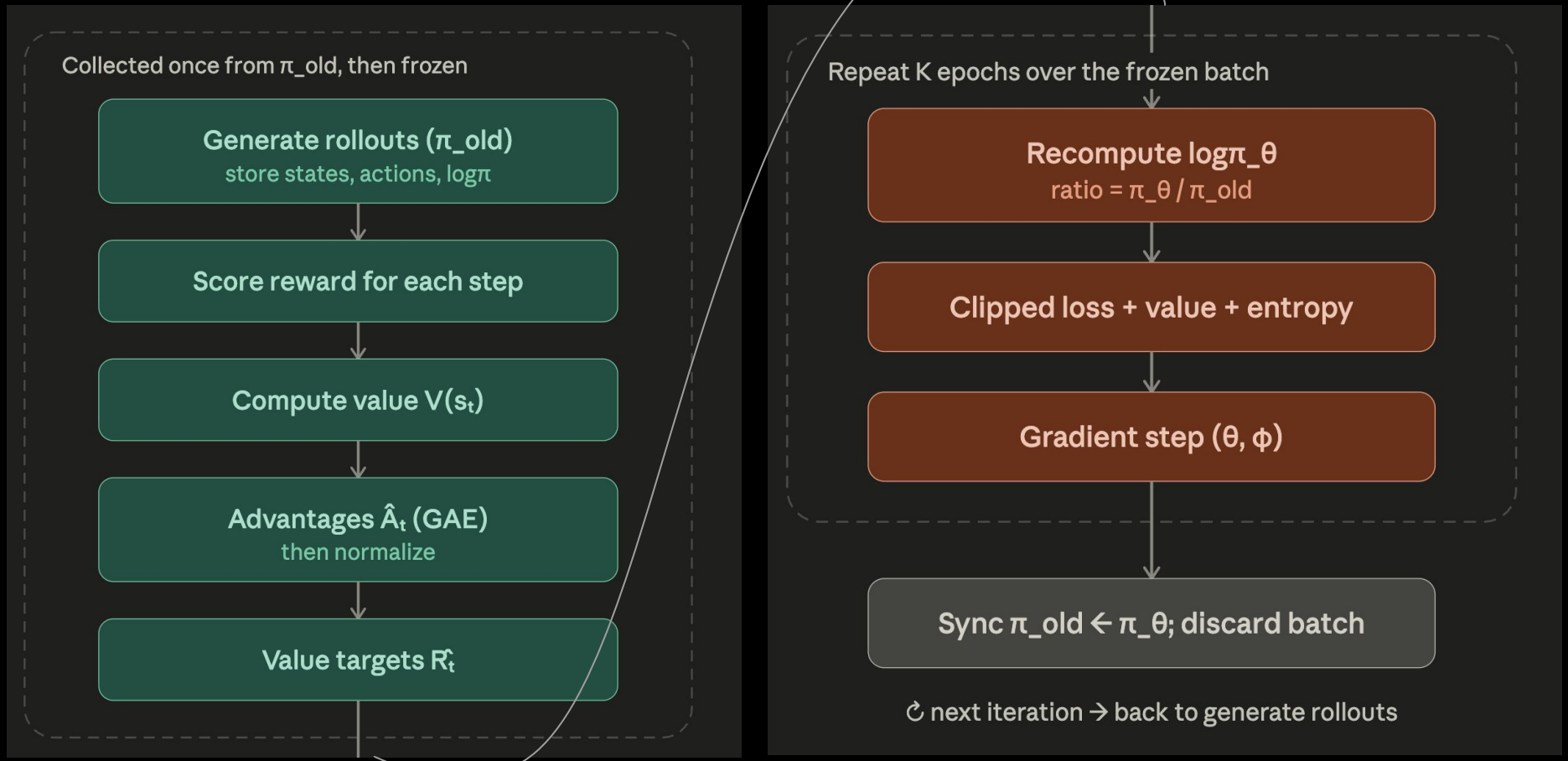
Objective

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \left\{ \frac{\pi_\theta(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t \right\}$$

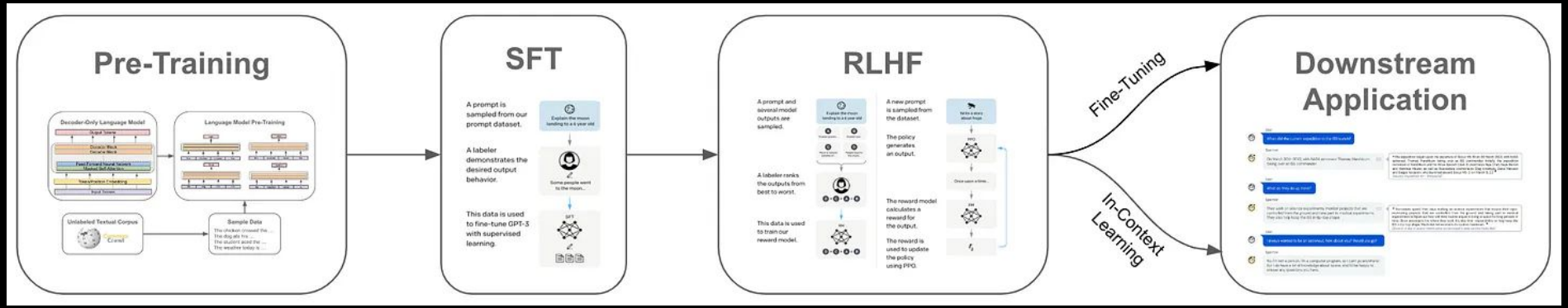
- Estimated advantage of current token.
- Level of agreement between reference-model and policy-model.



PPO Training Loop



End-to-End Pipeline



Any Questions?

